BLOCK 19

Dictionaries & Structured Records

Keys, values, and real-world data

“Look it up by name.”

```
record = {
  "record_id": "EQ-1045",
  "status": "active",
  "hours": 850
}
```

Instead of remembering positions,
we give values meaningful labels.

Today’s mental model

Dictionary = a labeled record
like a spreadsheet row, JSON object,
or API message.

BLOCK 19

Why dictionaries matter

Lists are great — until the meaning depends on memory.

```
record = ["EQ-1045", "active", 850, True]

print(record[2])
```

What is position 2?
Hours? Temperature? Errors?

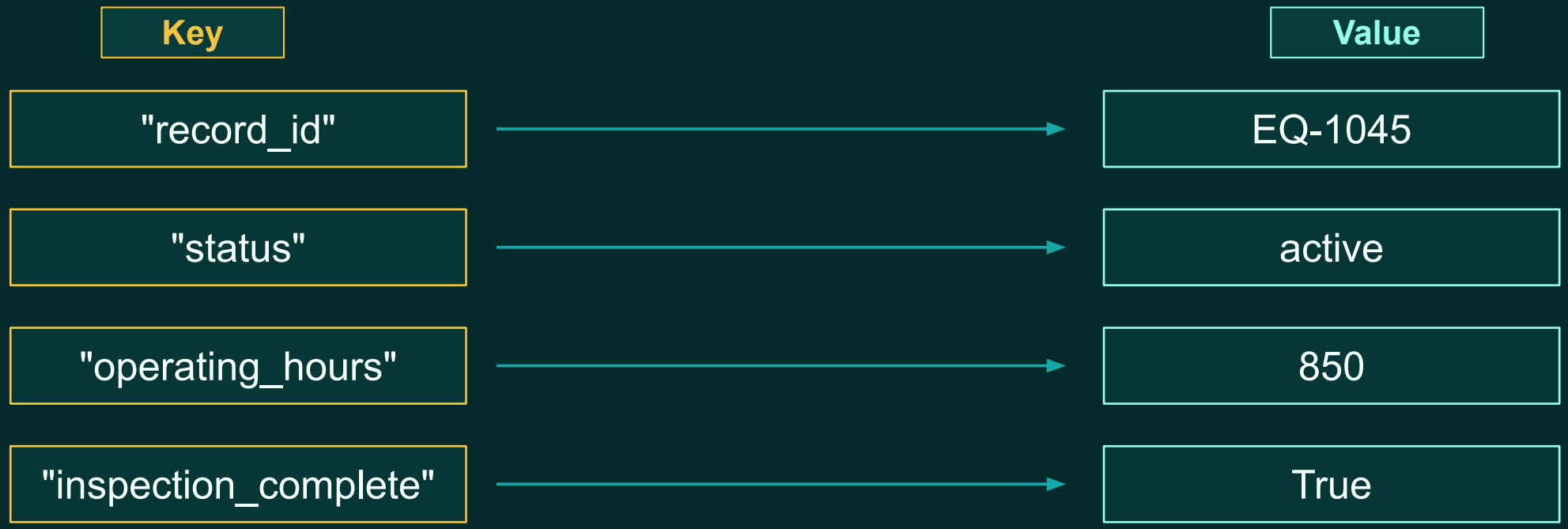
```
record = {
  "record_id": "EQ-1045",
  "status": "active",
  "operating_hours": 850,
  "inspection_complete": True,
}

print(record["operating_hours"])
```

A key explains what the value means.

BLOCK 19

From positions to names

A dictionary connects a key to a value.



The key is not just a label on a slide.
It is how the program finds the value.

BLOCK 19

Creating a dictionary

Curly braces hold named values.

```
equipment = {
  "equipment_id": "EQ-1045",
  "status": "active",
  "operating_hours": 850,
  "maximum_hours": 1000,
  "inspection_complete": True,
  "temperature": 72.5,
}
```

Syntax landmarks

- curly braces { }
- string keys
- colon between key and value
- comma between pairs
- mixed value types are allowed

Readable data beats mystery positions.

BLOCK 19

Retrieving values

Use the key inside square brackets.

```
print(equipment["equipment_id"])
print(equipment["status"])
print(equipment["temperature"])
```

Output

```
EQ-1045
active
72.5
```

Mental model

Dictionary lookup is like asking:
“Give me the value stored under this label.”

BLOCK 19

Updating and adding values

Dictionaries are mutable: the record can change.

```
equipment["status"] = "inactive"
```

Updates an existing key

```
equipment["error_count"] = 0
```

Adds a new key if it does not exist

Same square bracket syntax.
Different result depends on whether the key already exists.

BLOCK 19

Removing keys and checking size

Records can also lose fields.

```
removed_value = equipment.pop("temperature")

print(removed_value)
print(len(equipment))
```

.pop() removes a key and returns its value.

len(dictionary) tells how many key-value pairs are present.

Common use

Removing temporary fields, optional fields, or data that should not be carried forward.

BLOCK 19

Checking for keys

Before you look up a value, check that it exists.

```
has_status = "status" in equipment

if has_status:
    print(equipment["status"])
```

The in operator checks keys, not values.

```
"active" in equipment
```

False in most cases:
"active" is a value, not a key.

BLOCK 19

Safe retrieval with .get()

Two ways to ask for a value.

```
equipment["error_count"]
```

Use when the key should exist.

```
equipment.get("error_count")
```

Returns None if the key is missing.

```
equipment.get("error_count", 0)
```

Returns 0 if the key is missing.

Default values are useful for optional fields.

BLOCK 19

Dictionary methods

Three views of the same record.

```
record.keys()
record.values()
record.items()
```

keys()	values()	items()
field names	stored data	key-value pairs

You will use these heavily with loops.

BLOCK 19

Dictionaries as structured records

This looks like real data.

```
employee = {
  "employee_id": "EMP-203",
  "name": "Jordan Smith",
  "department": "engineering",
  "active": True,
}
```

Same idea appears as:

- a row in a spreadsheet
- a JSON object
- a database record
- a message from an API

Dictionaries are a bridge from beginner Python
to real data-processing work.

BLOCK 19

A spreadsheet row becomes a dictionary

Column names become keys.

employee_id	name	department	active
EMP-203	Jordan Smith	engineering	True

```
employee = {
  "employee_id": "EMP-203",
  "name": "Jordan Smith",
  "department": "engineering",
  "active": True,
}
```

A dictionary can represent one row.
A list of dictionaries can represent many rows.

BLOCK 19

Nested data structures

A dictionary can contain another dictionary.

```
record = {
  "record_id": "EQ-1045",
  "status": "active",
  "metrics": {
    "temperature": 72.5,
    "operating_hours": 850,
  },
}
```

```
temperature = record["metrics"]["temperature"]
```

Read from left to right:

record → metrics → temperature

Nested data shows up in APIs and JSON files.

BLOCK 19

Readiness checks inside a dictionary

Use named fields to make decisions.

```
equipment["is_ready"] = (
  equipment["status"] == "active"
  and equipment["inspection_complete"]
  and equipment["operating_hours"]
      <= equipment["maximum_hours"]
)
```

Result stored back into the same record.

The dictionary can hold both raw fields and derived fields.

This is the same validation logic from Block 16 — now attached to a structured record.

BLOCK 19

Guided exercise 1: equipment record

Create, read, update, and add fields.

```
equipment = {
  "equipment_id": "EQ-1045",
  "status": "active",
  "operating_hours": 850,
  "maximum_hours": 1000,
  "inspection_complete": True,
  "temperature": 72.5,
}
```

Student tasks

- print the equipment ID
- print current status
- update operating hours
- add error_count
- add is_ready

Exercise time: 8 minutes

BLOCK 19

Guided exercise 1: readiness field

Print every intermediate idea.

```
is_active = equipment["status"] == "active"
has_inspection = equipment["inspection_complete"]
is_under_limit = (
  equipment["operating_hours"]
  <= equipment["maximum_hours"]
)

equipment["is_ready"] = (
  is_active and has_inspection and is_under_limit
)
```

Readable debugging pattern

- name the checks
- print the checks
- combine the checks
- store the result

Clear code is easier to verify.

BLOCK 19

Guided exercise 2: project record

Calculate and add a derived value.

```
project = {
  "project_id": "PRJ-2401",
  "name": "Network Modernization",
  "completed_tasks": 18,
  "total_tasks": 24,
  "approved": True,
}

project["completion_percentage"] = (
  project["completed_tasks"]
  / project["total_tasks"]
  * 100
)
```

Then print:

- project name
- completion percentage
- approval status
- the whole dictionary

Derived values become part of the record.

BLOCK 19

Guided exercise 3: missing keys

Predict before running.

```
print(project.get("budget"))
print(project.get("budget", 0))
print(project["budget"])
```

Prediction

1. None
2. 0
3. KeyError

Discussion

When should missing data be allowed?
When should it stop the program?

BLOCK 19

Applied challenge: build the record

Convert standalone variables into one dictionary.

```
record_id = "INGEST-2401"
file_name = "equipment.csv"
file_size_mb = 18.5
record_count = 12450
error_count = 2
approved = True
```

```
record = {
  "record_id": record_id,
  "file_name": file_name,
  "file_size_mb": file_size_mb,
  "record_count": record_count,
  "error_count": error_count,
  "approved": approved,
}
```

Step one: gather related facts
under meaningful keys.

BLOCK 19

Applied challenge: add derived fields

Now make the record useful.

```
record["supported_file_type"] = (
  record["file_name"].lower().endswith(".csv")
)

record["has_errors"] = record["error_count"] > 0

record["ready_for_processing"] = (
  record["supported_file_type"]
  and record["file_size_mb"] > 0
  and record["record_count"] > 0
  and record["approved"]
)
```

Derived fields answer questions:

- Is the file supported?
- Does it have errors?
- Can processing continue?
- What should a human see?

Readable keys make later code easier.

BLOCK 19

Common dictionary mistakes

Most errors are small spelling or syntax problems.

Missing comma

```
record = {
  "id": "EQ-1"
  "status": "active"
}
```

Wrong key name

```
record["status"]
record["Status"]
```

Expecting missing key

```
record["budget"]
```

Debugging move: print(record) and print(record.keys()).

BLOCK 19

Mini knowledge check

Answer in plain language first.

1. What is the difference between a key and a value?
2. When should you use .get()?
3. Why is a dictionary easier to read than a list for records?
4. What does record["metrics"]["temperature"] mean?

Predict → run → explain.

BLOCK 19

Wrap-up: choosing a collection

Pick the structure that matches the data.

List	Tuple	Dictionary	Coming next
Many values identified by order	Fixed group with meaning by position	Labeled fields inside a record	Loops will automate work across collections.

```
record["ready"] = True
print(record["ready"])
```

Dictionaries make code read like the data it describes.